哈爾濱工程大學

第 10 讲 / 全 17 讲

C 语言函数用法详解

把一段逻辑起个名字，写一次，用无数次

定义与调用

先把工具造好，再按名字拿去用

参数与返回

括号里送进去， return 送出来

值传递

形参是副本，改形参不动实参

库函数

math.h 里现成的工具直接用

主讲教师：郎大鹏

本节目标

哈爾濱工程大學

1. 说清为什么要写函数：消灭重复代码，改一处就全部生效
2. 会写函数的三行结构：返回类型、函数名与参数表、函数体
3. 分清有返回值的函数和 void 函数，并能正确调用它们
4. 理解值传递：形参只是实参的副本，改形参影响不到实参
5. 理解局部变量与作用域：函数里的变量只在自己的花括号内有效
6. 能用 scanf 、 for 和函数配合，独立完成本讲 5 道例题

01

函数入门

为什么要有函数 · 怎么定义 · 怎么调用 · 有返回值还是 void

哈爾濱工程大学

为什么需要函数

哈爾濱工程大學

```c
// 求 1 加到 100
sum = 0;
for (i = 1; i <= 100; i++) {
    sum = sum + i;
}
printf("%d", sum);  // 5050
// 求 1 加到 50：整段重抄
sum = 0;
for (i = 1; i <= 50; i++) {
    sum = sum + i;
}
printf("%d", sum);  // 1275
```

```c
// 写成函数，只写一次
int sumTo(int n) {
    int s = 0;
    int i;
    for (i = 1; i <= n; i++) {
        s = s + i;
    }
    return s;
}
printf("%d", sumTo(100));  // 5050
printf("%d", sumTo(50));  // 1275
```

① 不重复
写一次多调用

② 好读
见名就知意

③ 好改
改一处就够

④ 可复用
别处也能用

定义与调用：跳转与返回

```c
#include <stdio.h>
// ① 定义：把工具造好
int add(int a, int b) {
    return a + b;          // ③ 交出 8
}
int main(void) {
    int x;
    x = add(3, 5);         // ② 跳进去
    printf("%d", x);       // ④ 输出 8
    return 0;
}
```

① 定义 = 造工具

把一段逻辑写在 main 外面，起个名字叫 add ，以后就按这个名字找它

② 调用 = 拿来用

写 add(3, 5) 就是调用：程序立刻跳到 add 的第一行， main 原地暂停

③ 返回 = 带结果回来

add 执行到 return ，把 8 交回去， main 从断点的那一行接着往下走

有返回值 与 void

有返回值的函数算出一个结果，调用处要把这个结果接住、接着用； void 函数只做事、不交值，只能单独写成一条语句。

```
// 有返回值：算完要交出一个值
int max2(int a, int b) {
    if (a > b) {
        return a;
    }
    return b;
}
// 在 main 里这样用：
m = max2(3, 7);       // 值要接住
printf("%d", m);        // 输出 7
```

```
// void：只做事，不交值
void printLine(void) {
    int i;
    for (i = 0; i < 10; i++) {
        printf("-");
    }
}
printLine();              // 单独成一句
// 在 main 里这样用：
x = printLine();          // 编译报错
```

函数定义的三行结构

```
// ①  返回类型   ②  函数名   ③  参数表
int sum(int a, int b)
{                                  // ④  函数体
    int s;                         //  局部变量
    s = a + b;                     //  真正干活
    return s;                      //  送出去
}
//  拆开看就这三样东西：
//    int                  返回类型
//    sum                  函数名
//    (int a, int b)        参数表
//    { ... }               函数体
```

① 返回类型

函数交出来的值是什么类型；一个值都不交，就写 void

② 函数名与参数表

名字按用途起；参数要逐个写类型，一个都不需要就写 void

③ 函数体

一对花括号包住要执行的语句，这里是函数真正干活的地方

声明、定义、调用的位置

```c
#include <stdio.h>
int add(int a, int b);  // ① 声明
int main(void) {
    int r;
    r = add(3, 5);        // ③ 调用
    printf("%d", r);     // 输出 8
    return 0;
}
int add(int a, int b) { // ② 定义
    return a + b;
}
// 顺序也可以是：定义 → 调用
// 绝不能：调用 → 定义（无声明）
```

① 声明（原型）

写在调用之前，只报名字和长相，末尾必须有分号

② 定义

写出函数体，是唯一的一份实现，末尾不加分号

③ 调用

只要在此之前见过声明或定义，就可以放心调用它

一个完整程序走一遍


```c
#include <stdio.h>
int sumTo(int n);          // ① 声明
int main(void) {
    printf("%d", sumTo(100)); // ③ 第 1 次
    printf("%d", sumTo(50));  // ④ 第 2 次
    return 0;
}
int sumTo(int n) {         // ② 定义
    int s = 0, i;
    for (i = 1; i <= n; i++) {
        s = s + i;
    }
    return s;
}
//  屏幕输出：5050 1275
```

编译：从上到下

编译器一行行往下读，所以 main 用到 sumTo 之前必须先见过它

运行：从 main 开始

不管函数写在前面还是后面，第一个被执行的永远是 main

函数可以调多次

sumTo 只写了一次，却被调用两次，两次各自独立算一遍

02

参数与作用域

值传递 · 形参与实参 · 局部变量 · 变量作用域 · 标准库函数

哈爾濱工程大学

值传递：形参只是副本

```
void swap(int a, int b) {
    int t;
    t = a;
    a = b;
    b = t;
    printf("%d %d", a, b); // 20 10
}
int main(void) {
    int x = 10, y = 20;
    swap(x, y);
    printf("%d %d", x, y); // 10 20
    return 0;
}
//  两次输出不一样!
```

① 形参是副本

调用时把 x 、 y 的值复制一份交给 a 、 b ，四个变量各存各的

② 改副本没用

swap 里换掉的是 a 、 b ，回到 main ， x 和 y 一动没动

③ 先记住这个坑

值传递只拿得到副本，想改实参就得拿到变量本身 —— 后面指针课会解决

形参与实参对照


```c
// 实参：调用处真正交出去的值
int main(void) {
    int x = 3, y = 5;
    int r;
    r = add(x, y);      // x、y 是实参
    printf("%d", r);  // 输出 8
    return 0;
}
// 实参有自己的名字和存储
```

```c
// 形参：定义处等着接收的变量
int add(int a, int b) {
    int s;
    s = a + b;          // 用副本计算
    return s;           // 交出 8
}
// 形参只在函数体内存在
// 拿到的是实参复制来的值
// 名字不同，完全没关系
```

个数要相同

两个实参就配两个形参，多一个少一个都报错

类型要对应

int 实参配 int 形参，类型不一致会悄悄算错

顺序要对上

第一个实参给第一个形参，按位置一对一传递

局部变量：各自独立

```
void f1(void) {
    int k = 10;       // f1 的 k
    k = k + 1;
    printf("%d", k);  // 输出 11
}
void f2(void) {
    int k = 100;      // f2 的 k
    printf("%d", k);  // 输出 100
}
int main(void) {
    f1();
    f2();
    f1();             // k 又从 10 开始
    return 0;
}
```

① 同名不冲突

两个函数里都叫 k，却是两个完全不同的变量，各存各的值

② 进来才存在

函数被调用时变量才诞生，返回之后立刻消失得干干净净

③ 每次重新生

第二次调用 f1，k 又从 10 开始，它不记得上次的 11

变量作用域：花括号之间

```
int main(void) {
    int a = 1;          // a 从这里生效
    if (a > 0) {
        int b = 2;      // b 只在这对 { } 内
        printf("%d", a + b); // 输出 3
    }
    printf("%d", a);    // 输出 1，a 还在
    // printf("%d", b); // 错：b 已失效
    return 0;
}                       // a 到这里结束
// 形参也是局部变量
int add(int x) {        // x 的作用域
    return x + 1;       // 只在函数体内
}                       // 出函数就没了
```

① 从定义到花括号结束

一个变量只在它所属的那对花括号之内有效，出了就作废

② 内层看得到外层

if 的花括号里照样能用外面定义的 a，里外相加没问题

③ 外层看不到内层

出了 if，里面的 b 就消失了，再拿它用会直接编译报错

标准库函数： math.h

```c
#include <stdio.h>
#include <math.h>
int main(void) {
    double a;
    a = sqrt(9.0);        //  开平方
    printf("%.1f", a);   //  输出 3.0
    printf("%.1f", pow(2.0, 3.0)); // 8.0
    printf("%.1f", fabs(-7.5));  // 7.5
    return 0;
}
//  库函数和自己写的函数
```

sqrt(x)

求平方根； sqrt(9.0) 得到 3.0 ，参数和返回值都是 double

pow(x, y)

求 x 的 y 次方； pow(2.0, 3.0) 得到 8.0 ，要两个参数

fabs(x)

求绝对值； fabs(-7.5) 得到 7.5 ，负号直接去掉

用库函数前先 #include 对应的头文件： stdio.h 管输入输出， math.h 管数学函数

return 的两个作用

```
int isEven(int n) {
    if (n % 2 == 0) {
        return 1;       // ① 交出结果
    }
    return 0;           // ② 顺手结束
}
void check(int n) {
    if (n < 0) {
        return;         // 提前收工
    }
    printf("%d", n);   // 负数走不到
}
// 调用看看：
printf("%d", isEven(4));  // 输出 1
```

① 交出结果

return 加一个表达式，把值送回调用处，类型要和返回类型对上

② 立刻结束

一执行到 return 函数马上收工，它后面的语句永远不会被执行

③ void 也能用

void 函数里写光秃秃的 return; 不带值，只用来提前退出

03

综合运用

封装判断 · 多参数 · 函数调函数 · 用函数重写程序 · 调试视角

哈爾濱工程大学

封装判断：返回 1 / 0

```
int isPositive(int n) {
    if (n > 0) {
        return 1;      // 是：交回 1
    }
    return 0;          // 否：交回 0
}
int main(void) {
    int x;
    scanf("%d", &x);   // 输入一个数
    if (isPositive(x) == 1) {
        printf("正数");
    } else {
        printf("非正数");
    }
    return 0;
}
```

约定：is 开头

判定函数统一用 is 开头命名，成立交回 1，不成立交回 0

直接放进 if

isPositive(x) 当条件用，读起来就像在问一句话

好处：改一处

判定规则变了只改函数，主程序那几行一个字都不用动

多参数函数


```c
// 三个参数都要写类型
int total(int a, int b, int c) {
    int s;
    s = a + b + c;
    return s;
}
// 返回类型也可以不一样
double avg(int a, int b, int c) {
    return (a + b + c) / 3.0;
}
int main(void) {
    printf("%d", total(90, 88, 95)); // 273
    printf("%.1f", avg(90, 88, 95));  // 91.0
    return 0;
}
```

① 想给几个写几个

参数之间用逗号隔开，每一个都必须单独写上自己的类型

② 类型可以不同

int 、 double 混着写完全没问题，参数各写各的类型

③ 顺序不能乱

实参按位置一对一传进去，第一个实参给第一个形参

函数调用函数

```c
int square(int n) {      //  求平方
    return n * n;
}
//  函数里再调用另一个函数
int sumOfSquare(int a, int b) {
    return square(a) + square(b);
}
int main(void) {
    int r;
    r = sumOfSquare(3, 4); // 9 + 16
    printf("%d", r);       //  输出  25
    return 0;
}
//  调用链:  main → sumOfSquare → square
```

① 函数里照样能调

只要被调用的那个函数在此之前有声明或定义就行

② 一层套一层

main 调 sumOfSquare ，它再调 square ，链条想多长就多长

③ 每个各管一段

平方归 square ，相加归 sumOfSquare ，谁也不干扰谁

用函数重写以前的程序

哈爾濱工程大學

```c
// 以前：逻辑全堆在 main 里
int main(void) {
    int n, i, s = 0;
    scanf("%d", &n);
    for (i = 1; i <= n; i++) {
        s = s + i;
    }
    printf("%d", s);
    if (n % 2 == 0) {
        printf("%d", 1);
    } else {
        printf("%d", 0);
    }
    return 0;
}
```

```c
// 重写后：main 只管调度
int main(void) {
    int n;
    scanf("%d", &n);  // 输入
    printf("%d", sumTo(n)); // 求和
    printf("%d", isEven(n));// 判奇偶
    return 0;
}
```

int sumTo(int n)

累加求和，返回 1+2+…+n

int isEven(int n)

偶数返回 1 ，奇数返回 0

调试视角：执行顺序

```
int twice(int n) {
    return n * 2;
}
int main(void) {
    int a = 3;          // ①
    int b;
    b = twice(a);       // ② 断点
    printf("%d", b);    // ⑥ 输出 6
    return 0;
}
// 单步执行看清楚的六步：
//   main → twice → main
// 调试器里看到的调用栈，
// 就是 " 谁在等谁 " 的那条链
```

1. main 开始，a 得到 3
2. 走到 twice(a)，main 暂停
3. 跳进 twice，n 拿到副本 3
4. 执行 return，算出结果 6
5. 回到断点，b 得到 6
6. printf 输出 6，main 结束

函数设计小结

哈尔滨工程大学

一个函数只干一件事，名字要能说清它干了什么。做到这一点，程序就自然分成一块一块，读起来、改起来都轻松。

① 一个函数一件事

只做一件事，名字才起得出来。发现一个函数里干了两件事，就该拆成两个函数，让 main 去把它们串起来。

② 名字要会说话

求和叫 sumTo 、判素数叫 isPrime ，一眼就知道干什么。别用 f1 、 func 、 aaa 这种看不出意思的名字。

③ 参数越少越好

参数越多越难记、越容易传错位置。超过三四个的时候，先想想是不是该把这个函数拆开。

④ 只认参数和 return

需要的都从参数进来，结果都由 return 出去。这样函数拿到哪儿都能用，跟外面互不牵扯。

例 1 · 封装求和 sumTo(n)


```c
#include <stdio.h>
int sumTo(int n) {        // 只写一次
    int i, s = 0;
    for (i = 1; i <= n; i++) {
        s = s + i;
    }
    return s;
}
int main(void) {
    int a, b;
    a = sumTo(100);       // 第 1 次调用
    b = sumTo(50);        // 第 2 次调用
    printf("%d %d", a, b); // 5050 1275
    return 0;
}
```

题目

求 1+2+…+n ，把累加过程封装成函数，并在 main 里调用两次

运行结果

```
sumTo(100) = 5050
sumTo(50)  = 1275
屏幕: 5050 1275
```

要点

函数体只写了一次，却被调用两次

想算到 200 ，直接写 sumTo(200)

公式要改，只改函数体那一处

例 2 · 素数判定 isPrime


```c
#include <stdio.h>
int isPrime(int n) {
    int i;
    if (n < 2) {
        return 0;          // 2 以下不是素数
    }
    for (i = 2; i < n; i++) {
        if (n % i == 0) {
            return 0;      // 有因子，立刻退出
        }
    }
    return 1;              // 都除不动
}
// 调用: isPrime(17) 得到 1
// 调用: isPrime(18) 得到 0
```

题目

判断一个正整数是不是素数，是素数返回 1 ，不是返回 0

运行结果

```
isPrime(17) = 1
isPrime(18) = 0
isPrime(1)  = 0
```

要点

判定函数：成立返回 1 ，否则返回 0

找到因子立刻 return 0 ，不再白算

for 跑完都没整除，才轮到 return 1

例 3 · 最大公约数 gcd


```c
#include <stdio.h>
int gcd(int a, int b) {  // 两个参数
    int i;
    for (i = a; i >= 1; i--) { // 从大到小
        if (a % i == 0) {     // 先整除 a
            if (b % i == 0) { // 再整除 b
                return i;    // 找到就退出
            }
        }
    }
    return 1;              // 兜底: 1
}
// 调用: gcd(12, 18) 得到 6
// 两个参数 + 嵌套 if, 不用逻辑与
```

题目

求两个正整数的最大公约数，用 for 从大到小试除

运行结果

```
gcd(12, 18) = 6
gcd(20, 30) = 10
gcd(7, 5)   = 1
```

要点

从 a 往下试，第一个同时整除的就是答案

两层 if 嵌套：先判 a，再判 b

一找到就 return，后面的数不用再试

例 4 · 闰年封装 isLeap


```c
#include <stdio.h>
int isLeap(int year) {
    if (year % 4 == 0) {
        if (year % 100 == 0) {  // 整百年
            if (year % 400 == 0) {
                return 1;   // 400 的倍数
            }
            return 0;       // 不满 400
        }
        return 1;           // 普通闰年
    }
    return 0;               // 不被 4 整除
}
// 调用: isLeap(2000) 得到 1
// 调用: isLeap(1900) 得到 0
```

题目

把 if 课学过的闰年规则封装成函数，是闰年返回 1，不是返回 0

运行结果

```
isLeap(2000) = 1
isLeap(1900) = 0
isLeap(2024) = 1
```

要点

先看能不能被 4 整除，再看是不是整百年

整百年必须能被 400 整除才算闰年

每层 if 都有自己的 return，层层筛选

例 5 · 平均值 average(n)

```c
#include <stdio.h>
double average(int n) {  // n 是数据个数
    int i;
    double x, sum = 0;
    for (i = 1; i <= n; i++) {
        scanf("%lf", &x);  // 边读边加
        sum = sum + x;
    }
    return sum / n;        // 总和除以个数
}
int main(void) {
    printf("%.2f", average(3));
    return 0;
}
```

题目

读入 n 个整数求平均值，把 scanf、for 累加和函数封装串起来

运行结果

```
输入: 90 80 70
average(3) = 80.00
屏幕: 80.00
```

要点

scanf 读进来，for 累加，函数负责算

n 从参数进来，结果由 return 交出去

主程序只剩输入、调用、输出三件事

常见错误盘点

六个高频坑

哈尔滨工程大学

✗ 声明忘了分号： int add(int a, int b) 后面少了那个分号
声明末尾必须加分号，定义后面直接跟花括号

✗ 形参类型漏写： add(int a, b) 里的 b 没有类型
每个参数都要单独写类型： add(int a, int b)

✗ 想靠形参改实参：改的是副本，实参一动不动
想真改就得拿到变量本身 —— 这个坑指针课会解决

✗ return 后面还写代码，以为函数会继续执行下去
return 之后的语句永不到达，要提前返回就放进 if

✗ 函数名和变量名重名： int max; 又写 int max(...)
名字各起各的，别让编译器分不清谁是谁

✗ 没声明就调用：调用写在了函数定义的前面
先补一句声明，或把整个定义挪到调用之前

小结与预告

函数是给一段逻辑起个名字：写一次，调用无数次。
参数进来、 return 出去；形参只是副本，改不动实参。
一个函数只干一件事，名字要能说出它干了什么。

下节预告

swap 失败之谜，指针课揭晓

今天我们只能让函数拿到值的副本，所以 swap 怎么也换不动 x 和 y。等学完指针，让函数直接找到变量本身，交换就成功了。

哈尔滨工程大学